

Script-Controlled ACL

Restrict Record Access Based on Field Value

SCENARIO Creating roles and ACLs, then accessing Institution Details records based on role and Branch	Entice How does someone initially become aware of this process?	Enter What do people experience as they begin the process?	Engage In the core moments in the process, what happens?	Exit What do people typically experience as the process finishes?	Extend What happens after the experience is over?
Steps What does the person (or group) typically experience?	Spot the need Admin notices student records in u_institution_details are visible to everyone and need protection. Plan the ACL Admin decides to use a script-controlled ACL that checks the user's role and the Branch field.	Create user & roles Admin creates the EEE User and roles bb1, bb2, bb3 and bb4, and assigns them to the user. Create table & records Admin creates u_institution_details with its fields and adds ECE, EEE and CSE records. Elevate role Admin elevates to security_admin and opens System Security > Access Control (ACL).	Create Read ACL Admin sets Advanced = true, role bb1, data condition Branch is EEE, and adds the script. Create, Write & Delete ACLs Admin creates ACLs requiring bb2, bb3 and bb4, with no data condition. Open the list User opens u_institution_details.list; the ACL script runs and returns true or false.	See the result bb1 user sees only EEE records; a user without the role sees none; admin sees all records. Create, edit & delete Users with bb2, bb3 and bb4 can add, edit and delete EEE records.	Verify access Admin impersonates the EEE User and Admin to confirm each result. Extend the rules Admin adds more branches, roles or script conditions as needs grow.
Interactions What interactions do they have at each step along the way?	Records list in the ServiceNow web interface Admin and project team (planning)	User Administration > Users and Roles (ServiceNow web UI) System Definition > Tables; Institution Details list Profile menu > Elevate role; Access Control (ACL) list	ACL form: Type, Operation, Name, Requires role, Data Condition and Script editor ACL form and Requires role related list u_institution_details.list in the All menu while impersonating a user	Institution Details list view (EEE User, user without role, Admin) New button, inline edit, and Actions on selected rows > Delete	Impersonate user option in the profile menu Roles, ACL rules and table fields
Goals & motivations At each step, what is a person's primary goal or motivation? ("Help me..." or "Help me avoid...")	Help me keep student data from being seen by the wrong users Help me choose a simple rule that works for roles and branch	Help me give each user only the roles they need Help me have realistic data to test with Help me get permission to edit ACLs	Help me make sure bb1 users see only EEE records Help me control who can add, edit and delete Help me see the right records without extra steps	Help me trust that my access is correct Help me do my job without being blocked	Help me confirm the ACLs work before sharing Help me adapt the rules without starting over
Positive moments What steps does a typical person find enjoyable, productive, fun, motivating, delightful, or exciting?		Roles are quick to create using the same steps.	The script is short and easy to read (admin, bb1, otherwise false).	The result is clear: the right records, none, or all. Each action is controlled by its own role.	Impersonation makes testing easy without logging out.
Negative moments What steps does a typical person find frustrating, confusing, angering, costly, or time-consuming?		Forgetting to elevate security_admin blocks ACL changes.	A script that returns false hides all records, even from the right users. Role names (bb1 to bb4) are easy to mix up.	A user without the role sees an empty list with no explanation.	More branches mean more rules to keep updated.
Areas of opportunity How might we make each step better? What ideas do we have? What have others suggested?	How might we reuse one standard ACL pattern for other tables?	How might we remind admins to elevate before opening ACLs?	How might we test the script before saving it? Could clearer role names (e.g. eee_read) replace bb1 to bb4?	How might we show a helpful message instead of an empty list?	Could groups or a branch field on the user replace hard-coded roles?